ESCUELA COLOMBIANA DE INGENIERÍA
DESARROLLO ORIENTADO POR OBJETOS
Persistencia
2026-2 — Laboratorio 6/6 [:)]

OBJETIVO

Evidenciar el logro de los siguientes resultados de aprendizaje:

1. Extender el código de un proyecto considerando nuevos requisitos funcionales.
2. Diseñar y construir los métodos básicos de manejo de archivos: guardar como, abrir, exportar como e importar.
3. Controlar las excepciones generadas al trabajar con archivos.
4. Experimentar las prácticas

   Coding Only one pair integrates code at a time.
   Coding. Use collective ownership.

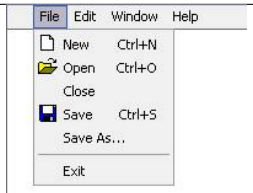
ENTREGA

- Incluyan en un archivo `.zip` los archivos correspondientes al laboratorio. El nombre debe ser los apellidos de los dos miembros del equipo ordenados alfabéticamente y separados por un guion. (CasallasC-DuarteJ)
- Publiquen el avance al final de la sesión y la versión definitiva en la fecha indicada, en los espacios correspondientes.

CONTEXTO

Reto: Extender el proyecto `safari` adicionando el menú `Archivo` en el menú barra con las opciones básicas de `entrada-salida (Abrir - Guardar como e Importar - Exportar como)` y las opciones estándar `Nuevo` y `Salir`.

El menú `Archivo` reúne las funciones básicas de gestión de documentos organizadas de forma consistente y predecible.
Las guías de *Apple* y las directrices de *Microsoft* para el diseño de interfaz establecen que esta estructura y semántica deben mantenerse estables para garantizar la usabilidad, constituyendo así un estándar de facto en las interfaces gráficas modernas.



PARTE I. Creando la maqueta [En lab06.pdf, safari.astah, safari.*.java]

(NO OLVIDEN BDD – MDD)

En este punto vamos a construir la maqueta correspondiente a esta extensión siguiendo el patrón MVC.

A `safari` inicial

1. En su directorio descarguen el laboratorio 03 realizado por ustedes y seleccionen el ambiente para trabajar: `BlueJ` o `VS Code`. ¿Cuál ambiente seleccionaron? ¿Por qué?
2. Ejecuten el programa y capturen la pantalla.

B Maqueta: MVC

1. **MODELO:** Preparen en la clase fachada del dominio los métodos correspondientes a las cuatro opciones básicas de entrada-salida (`open`, `saveAs`, `import_` y `exportAs`). Estos métodos deben tener un parámetro `File` y simplemente deben propagar una `EcoSafari Exception` con el mensaje de "Opción `nombreOpción` en construcción. Archivo `nombreArchivo`".

2. **VISTA :** Construyan el menú `Archivo` que ofrezca, además de las opciones básicas de entrada-salida, las opciones estándar de nuevo y salir (`Nuevo, Abrir, Salvar como, Importar, Exportar como y Salir`). No olviden incluir los separadores.
   Para esto creen el método `prepareViewMenu`. Capturen la pantalla del menú.
3. **CONTROLADOR:** Construyan los oyentes correspondientes a las seis opciones. Para esto creen el método `prepareControllerMenu` y los métodos base del controlador (`optionOpen`, `optionSaveAs`, `optionImport`, `optionExportAs`, `optionNew`, `optionExit`).
   a) `Salir`: El método `optionExit` que hace que se termine la aplicación. No es necesario incluir confirmación.
   b) `Nuevo`: El método `optionNew` que crea un nuevo `safari`. Capturen una pantalla significativa.
   c) `Entrada-salida`: Los métodos usan un `FileChooser` y atienden la excepción. Estos métodos llaman el método correspondiente del modelo que, por ahora, sólo lanzan una excepción. Ejecuten las diferentes acciones del menú y para cada una de ellas capture una pantalla significativa.

PARTE II. Implementando persistencia [En lab06.pdf, safari.astah, safari.*.java]

(NO OLVIDEN BDD – MDD)

A Salvar y Abrir

Las opciones guardar y abrir van a ofrecer servicios de persistencia del `safari` como objeto. Los nombres de los archivos deben tener como extensión `.dat`.

1. Copien las versiones actuales de `open` y `saveAs` y renómbrenlos como `open00` y `saveAs00`
   (No incluyan las versiones 00 en el diagrama de clases)
2. Construyan el método `saveAs` que ofrece el servicio de guardar en un archivo el estado actual de `safari`. Por ahora para las excepciones sólo consideren un mensaje de error general. No olviden diseño y pruebas de unidad.
3. Validen este método guardando el estado obtenido después de dos clics como `safariOne.dat`. ¿El archivo se creó en el disco? ¿Cuánto espacio ocupa?
4. Construyan el método `open` que ofrece el servicio de leer de un archivo. Por ahora para las excepciones sólo consideren un mensaje de error general. No olviden diseño y pruebas de unidad.
5. Realicen una prueba de aceptación para este método iniciando la aplicación, creando un nuevo estado y abriendo el archivo `safariOne.dat`. Capturen imágenes significativas de estos resultados.

B Exportar e Importar

Estas operaciones nos van a permitir importar información de `safari` desde un archivo de texto y exportarlo. Los nombres de los archivos de texto deben tener como extensión `.txt`. Los archivos texto tienen una línea de texto por cada ítem. En cada línea asociada un elemento se especifica el tipo y la posición.

```
Elephant 10, 20
Bush 15, 15
Storm 5, 5
```

Empleen los nombres de las clases definidos en su aplicación.

1. Copien las versiones actuales de `import_` y `exportAs` y renómbrenlos como `import00` y `exportAs00`.
   (No incluyan las versiones 00 en el diagrama de clases)
2. Construyan el método `exportAs` que ofrece el servicio de exportar a un archivo texto el estado actual, con el formato definido. Por ahora para las excepciones sólo consideren un mensaje de error general. No olviden diseño y pruebas de unidad.
3. Realicen una prueba de aceptación de este método: iniciando la aplicación y exportando como `safariOne.txt`. Editen el archivo y analicen los resultados. ¿Qué pasó?
4. Construyan el método `import_` que ofrece el servicio de importar de un archivo texto con el formato definido. Por ahora sólo considere un mensaje de error general. No olviden diseño y pruebas de unidad. (Consulten en la clase `String` los métodos `trim` y `split`)

5. Realicen una prueba de aceptación de este par de métodos: iniciando la aplicación exportando a `safariOther.txt`. saliendo, entrando, reiniciando (`Nuevo`) e importando el archivo `safariOther.txt`. ¿Qué resultado obtuvieron? Capturen la pantalla final.
6. Realicen otra prueba de aceptación de este método escribiendo un archivo de texto correcto en `safariOne.txt` e importe este archivo. ¿Qué resultado obtuvieron? Capturen la pantalla.
7. Eliminen el método que permitía adicionar los nuevos elementos en safari. A partir de ahora, la adición de elementos se realizará mediante el mecanismo de archivos.

C Comportamiento

1. Ejecuten la aplicación, den tres clics, guarden a un archivo cualquiera y ábranlo. Describan el comportamiento
2. Ejecuten la aplicación, tres clics, exporten a un archivo cualquiera e importen. Describan el comportamiento
3. ¿Qué diferencias ven el comportamiento 1. y 2.? Expliquen los resultados.

PARTE III. Fortaleciendo la persistencia [En lab06.pdf, safari.astah, safari.*.java]

(NO OLVIDEN BDD – MDD)

A Guardar y abrir

1. Copien las versiones actuales de `open` y `saveAs` y renómbrenlos como `open01` y `saveAs01`
   (No incluyan las versiones 01 en el diagrama de clases)
2. Perfeccionen el manejo de excepciones de los métodos `open` y `saveAs` detallando los errores. No olviden pruebas de unidad.
3. Realicen una prueba de aceptación para validar uno de los nuevos mensajes diseñados, ejecútenla y capturen la pantalla final.

B Exportar e Importar

1. Copien las versiones actuales de `import_` y `exportAs` y renómbrenlos como `import01` y `exportAs01`
   (No incluyan las versiones 01 en el diagrama de clases)
2. Perfeccionen el manejo de excepciones de los métodos `import_` y `exportAs` detallando los errores. No olviden pruebas de unidad.
3. Realicen una prueba de aceptación para validar uno de los nuevos mensajes diseñados, ejecútenla y capturen la pantalla final.

PARTE IV. BONO. Perfeccionando importar [En lab06.pdf, safari.astah, safari.*.java]

(NO OLVIDEN BDD – MDD)

A Minicompilador

1. Copien la versión actual de `import_` y renómbrenla como `import02`.
   (No incluyan la versión 02 en el diagrama de clases)
2. Perfeccionen el método `import_` para que, además de los errores generales, en las excepciones indique el detalle de los errores encontrados en el archivo (como un compilador) : número de línea donde se encontró el error, palabra que tiene el error y causa de error.
3. Escriban otro archivo con mínimo tres errores diferentes, llámenlo `safariErr.txt`, para ir arreglándolo con ayuda de su "importador". Presenten las pantallas que contengan los errores.

B Minicompilador flexible

1. Copien la versión actual de `import_` y renómbrenla como `import03`.
   (No incluyan la versión 03 en el diagrama de clases)
2. Perfeccionen el método `import_` para que pueda servir para cualquier tipo de elementos creados en el futuro. No olviden pruebas de unidad. (Investiguen cómo crear un objeto de una clase dado

su nombre)
3. El método `exportAs`, ¿necesita cambiar para ser flexible? Justifiquen la respuesta.
4. Si la respuesta anterior fue afirmativa, perfeccionen el método `exportAs`. No olviden desarrollar las pruebas de unidad correspondientes.
5. Escriban otro archivo de pruebas, llámenlo `safariFlex.txt`, para probar la flexibilidad. Presente una pantalla que contenga un error significativo.

RETROSPECTIVA

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas)
2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?
3. Considerando las prácticas XP del laboratorio, ¿cuál fue la más útil? ¿por qué?
4. ¿Cuál consideran fue el mayor logro? ¿Por qué?
5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?
6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?
7. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.